

HR Predictor Pro

What changed from your original files

Bugs fixed in `app.py` (was `updated_app.py`):

1. `except (... , IndexFormatter, ...)` — `IndexFormatter` isn't a real exception class; this would have crashed with a `NameError` the instant any request failed. Removed it.
2. The MLB stats URL was malformed: `https://mlb.com{player_id}/stats?...` is not a real endpoint. Replaced with the actual public MLB Stats API:
`https://statsapi.mlb.com/api/v1/people/{personId}/stats?stats=season&group=hitting&season=2026` — verified against MLB's own API docs and the open-source `MLB-StatsAPI` Python wrapper that uses this exact pattern.
3. Added a `/player_search` endpoint so the frontend isn't limited to a hardcoded roster.
4. Added graceful fallbacks everywhere a live API call can fail (player hasn't debuted yet, name typo, network hiccup, rate limiting) so the app never crashes — it just labels the response `data_source: "fallback_defaults"`.

Important honesty note: The free MLB Stats API gives real season totals (HR count, AVG, ERA, WHIP) but not per-pitch Statcast data (actual exit velocity/launch angle for a specific at-bat) — that requires a paid/partner feed. So the model derives plausible exit velocity / launch angle from real season power numbers using documented heuristics in `derive_exit_velocity()` / `derive_launch_angle()`. This is reasonable for a hobby app, but don't market it as "real Statcast data" since it isn't.

Files

- `app.py` — Flask backend. Trains the logistic regression on startup, exposes `/predict`, `/predict_live`, `/player_search`, `/health`.
- `requirements.txt` — pinned deps.
- `HRPredictorApp.jsx` — the web frontend (shown as an artifact). For your actual React Native app, port the same fetch calls into `app_p3.js`.

Running the backend locally

```
pip install -r requirements.txt
python app.py
```

Runs on `http://localhost:5001` . Test it:

```
curl -X POST http://localhost:5001/predict_live \
  -H "Content-Type: application/json" \
  -d '{"batter_id": 592450, "pitcher_id": 543037, "wind_speed": 8}'
```

Deploying so your phone/app can reach it

`localhost` only works from the same machine. For your React Native app on a physical phone or simulator, you need one of:

- **Same wifi network:** use your computer's local IP (e.g. `192.168.1.42:5001`) like your original `java script` file did — fine for development only.
- **Render / Railway / Fly.io:** deploy this Flask app for a stable public URL (your `app_p3.js` already references `onrender.com` as a placeholder — point it at your real deployed URL + `/predict_live`).

Wiring up Firebase auth for real

The artifact's auth screen is a working demo (any email/password "succeeds"). To make it real:

1. Fill in your actual config in `firebase.js` (the one you already wrote)
2. Swap the `setTimeout` mock in `AuthScreen` for the real

```
createUserWithEmailAndPassword / signInWithEmailAndPassword calls from
firebase/auth , exactly as in your app_p3.js
```

Known limitation in this chat

I can't make live network calls from this sandbox to verify `statsapi.mlb.com` responds exactly as documented right now — egress is disabled here. The endpoint pattern is corroborated by MLB's own Swagger docs and multiple independent open-source wrappers, but test it yourself with the `curl` command above once you run the backend on a machine with internet access.

